



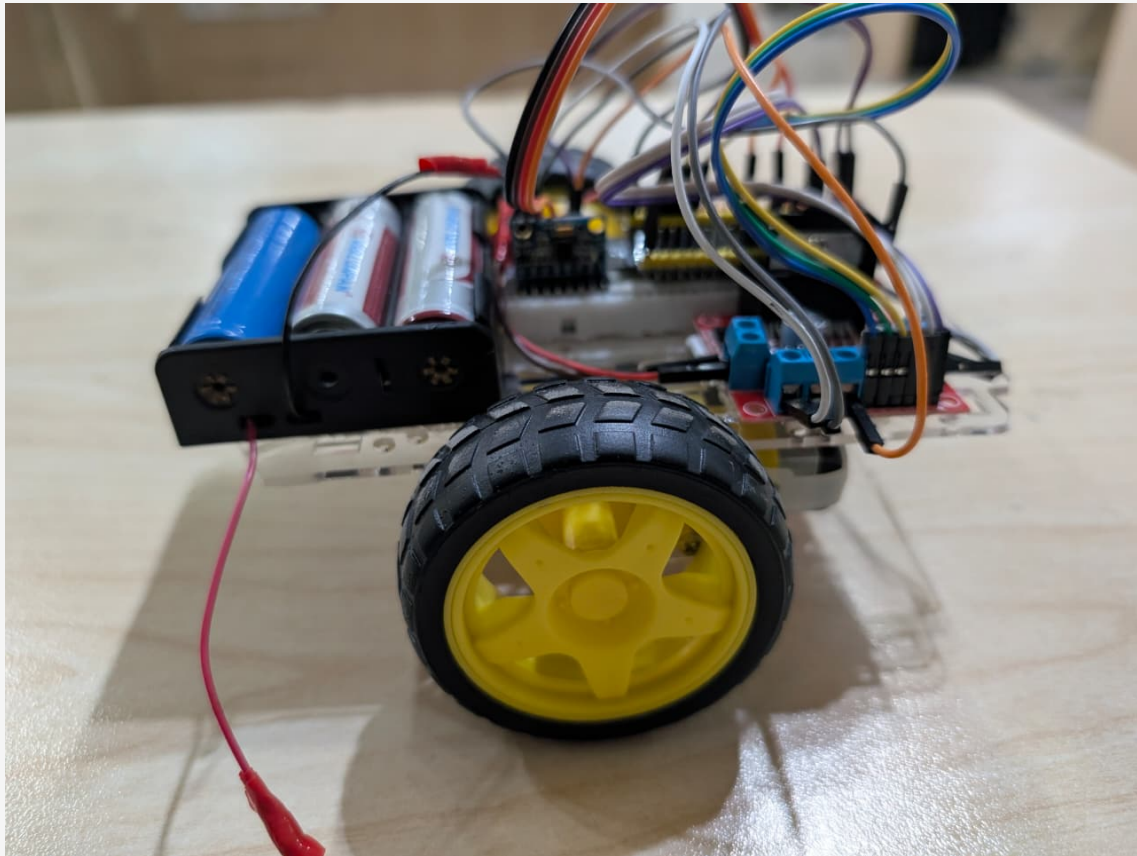
University of Engineering & Technology, Lahore

Department of Electrical Engineering

Control Systems Laboratory

Self-Balancing Two-Wheel Robot Using PID Control

Lab 12 — Complex Engineering Problem (CEP) – Part 1



Submitted By: Muhammad Mouzzam 2023-EE-111

Umair Nadeem 2023-EE-115

Submitted To: Dr. Habib Wajid

Prof. Hanan Tariq

Date: June 2, 2026

Contents

1	Objective	2
2	Background Theory	2
2.1	Inverted Pendulum Dynamics	2
2.2	Sensor Fusion	2
2.3	PID Control Law	3
3	Hardware Setup	3
3.1	Components List	3
3.2	Pin Connections	4
3.3	Power Architecture	4
4	IMU Interfacing and Angle Estimation	4
4.1	I ² C Initialization	4
4.2	Accelerometer Angle Calculation	4
4.3	Gyroscope Rate	5
4.4	Complementary Filter	5
5	PID Controller Design	5
5.1	Controller Structure	5
5.2	PID Gain Constants	5
5.3	Motor Command Mapping	6
6	Controller Tuning Procedure	6
7	Experimental Results	7
7.1	Tilt Angle Response	7
7.2	PWM / Motor Command Response	8
7.3	System Disturbance Rejection Response	8
8	Source Code	9
9	Discussion	11
10	Conclusion	11
	References	12

1. Objective

The objective of this laboratory experiment is to design and implement a PID (Proportional-Integral-Derivative) controller for stabilizing a self-balancing two-wheel inverted pendulum robot. Specifically, the experiment required us to:

- Interface an Inertial Measurement Unit (IMU) with a microcontroller over I²C.
- Estimate the real-time tilt angle of the robot using a complementary filter.
- Implement a discrete PID control loop running at 100.
- Drive two DC motors through an H-bridge motor driver.
- Achieve stable upright balancing and disturbance rejection.

2. Background Theory

2.1. Inverted Pendulum Dynamics

A two-wheeled self-balancing robot is mechanically equivalent to an inverted pendulum mounted on a mobile base. The system is inherently unstable: without active control, any small perturbation in tilt angle causes the robot to fall. The linearized dynamics around the upright equilibrium are given by:

$$\ddot{\theta} = a\theta + bu \tag{1}$$

where θ is the tilt angle, u is the motor torque input, and a , b are system parameters determined by physical characteristics (mass, moment of inertia, wheel radius). Because $a > 0$, the open-loop system has an unstable pole, and closed-loop feedback is essential.

2.2. Sensor Fusion

The MPU6050 provides both a 3-axis accelerometer and a 3-axis gyroscope. Each sensor has a complementary limitation:

- **Accelerometer:** gives an absolute angle estimate using gravity, but is noisy and susceptible to vibration.
- **Gyroscope:** measures angular rate precisely in the short term, but integrating the rate introduces drift over time.

A complementary filter fuses both:

$$\theta_k = \alpha(\theta_{k-1} + \omega \Delta t) + (1 - \alpha) \theta_{\text{acc}} \quad (2)$$

where α is the filter coefficient ($0 < \alpha < 1$), ω is the gyroscope reading in /s, and θ_{acc} is the accelerometer-derived angle.

2.3. PID Control Law

The PID controller computes a control signal $u(t)$ to drive the angle error $e(t) = \theta_{\text{desired}} - \theta(t)$ toward zero:

$$u(t) = K_p e(t) + K_i \int_0^t e(\tau) d\tau + K_d \frac{de(t)}{dt} \quad (3)$$

In discrete form with sample period Δt :

$$u[k] = K_p e[k] + K_i \sum_{j=0}^k e[j] \Delta t + K_d \frac{e[k] - e[k-1]}{\Delta t} \quad (4)$$

3. Hardware Setup

3.1. Components List

Table 1: Hardware Components Used

Component	Model / Specification	Role
Microcontroller	ESP32-S3 N16R8	Main processing unit, PID loop execution
IMU Sensor	GY-MPU6050	Accelerometer + Gyroscope (I ² C)
Motor Driver	L298N H-Bridge	Bidirectional DC motor control with PWM
DC Motors	2× Small DC Motors	Drive wheels
Power Supply	12 V Input	Powers motor driver; regulated down for MCU
Miscellaneous	Small Breadboard	Prototyping and signal routing

3.2. Pin Connections

Table 2: ESP32-S3 to L298N Motor Driver Pin Mapping

ESP32-S3 GPIO	L298N Pin	Function
GPIO 4	ENA	Motor A speed (PWM)
GPIO 5	IN1	Motor A direction
GPIO 6	IN2	Motor A direction
GPIO 7	IN3	Motor B direction
GPIO 12	IN4	Motor B direction
GPIO 13	ENB	Motor B speed (PWM)

Table 3: ESP32-S3 to MPU6050 I²C Pin Mapping

ESP32-S3 GPIO	MPU6050 Pin	Function
GPIO 8	SDA	I ² C Data
GPIO 9	SCL	I ² C Clock
3.3 V	VCC	Sensor power supply
GND	GND	Common ground

3.3. Power Architecture

The 12 V input supply feeds the L298N motor driver's VM rail directly. The ESP32-S3 and MPU6050 are powered via the L298N's onboard 5 V regulator output, stepped down further to 3.3 V by the ESP32-S3's internal regulator.

4. IMU Interfacing and Angle Estimation

4.1. I²C Initialization

The MPU6050 communicates at I²C address **0x68**. On startup, register **0x6B** (Power Management 1) is written with **0x00** to wake the device from its default sleep state.

4.2. Accelerometer Angle Calculation

The pitch angle is estimated from the raw accelerometer readings using:

$$\theta_{\text{acc}} = \arctan\left(\frac{a_y}{a_x}\right) \times \frac{180}{\pi} \quad (5)$$

4.3. Gyroscope Rate

Raw gyroscope data is converted to degrees per second using the sensitivity at the default ± 250 /s full-scale range (sensitivity: 131 LSB per /s):

$$\omega = -\frac{g_z}{131.0} \quad [^\circ/\text{s}] \quad (6)$$

The negative sign accounts for the physical mounting orientation of the sensor relative to the robot's tilt axis.

4.4. Complementary Filter

The fused angle is updated every control loop iteration at $\Delta t = 10$ ms (100 Hz):

$$\theta_k = 0.9 \times (\theta_{k-1} + \omega \Delta t) + 0.1 \times \theta_{\text{acc}} \quad (7)$$

The filter coefficient $\alpha = 0.9$ was selected after experimentation. It provides sufficient gyroscope weighting for smooth short-term tracking while the accelerometer prevents long-term drift. Higher values (e.g. 0.98) led to very slow correction of accumulated error; 0.9 offered a practical balance for this platform.

5. PID Controller Design

5.1. Controller Structure

The PID controller is implemented in discrete time within the main 100 Hz control loop. The reference setpoint is $\theta_{\text{desired}} = 0$ (upright position).

$$e[k] = 0 - \theta[k] \quad (8)$$

$$I[k] = I[k-1] + e[k] \Delta t \quad (9)$$

$$D[k] = \frac{e[k] - e[k-1]}{\Delta t} \quad (10)$$

$$u[k] = K_p e[k] + K_i I[k] + K_d D[k] \quad (11)$$

5.2. PID Gain Constants

The final tuned gain values are listed in Table 4. The tuning procedure and rationale are described in Section 6.

Table 4: PID Controller Gain Constants

Parameter	Symbol	Value	Description
Proportional Gain	K_p	45.0	Primary restoring force; drives motors proportional to tilt error
Integral Gain	K_i	25.0	Eliminates steady-state bias offset (e.g. centre-of-mass asymmetry)
Derivative Gain	K_d	0.2	Dampens oscillations by opposing the rate of change of error
Desired Angle	θ_d	0.0	Upright setpoint
Sample Period	Δt	0.01 s	100 Hz control loop

5.3. Motor Command Mapping

The control signal $u[k]$ is mapped to PWM duty cycles with a deadband floor to overcome motor stiction:

$$\text{PWM} = \text{clamp}(\text{MIN_PWM} + |u[k]|, 0, 511) \quad (12)$$

where $\text{MIN_PWM} = 90$ (9-bit resolution, 15 kHz frequency). If $u > 0$, both motors drive forward; if $u < 0$, both motors drive backward; if $u = 0$, motors coast.

6. Controller Tuning Procedure

The gains were tuned manually using the classical iterative approach, starting from zero integral and derivative action:

Step 1 — Proportional Gain (K_p)

With $K_i = 0$ and $K_d = 0$, K_p was increased from zero until the robot actively reacted to tilts and attempted to self-correct. Too low a value caused sluggish, non-recovering behaviour; too high caused violent oscillation. A value of $K_p \approx 45$ gave responsive but controllable correction.

Step 2 — Derivative Gain (K_d)

K_d was added to damp the oscillations introduced by proportional action. A deliberately small value of $K_d = 0.2$ was chosen. Because K_d was kept low, the robot exhibited almost

no jitter or chattering—the small gain provided just enough damping without amplifying sensor noise. This is a practical advantage of conservative derivative tuning.

Step 3 — Integral Gain (K_i)

K_i was introduced last and increased slowly to correct the residual lean caused by centre-of-mass offset and mechanical asymmetry. A value of $K_i = 25$ was found to eliminate the steady-state error without inducing integral windup or oscillation.

7. Experimental Results

7.1. Tilt Angle Response

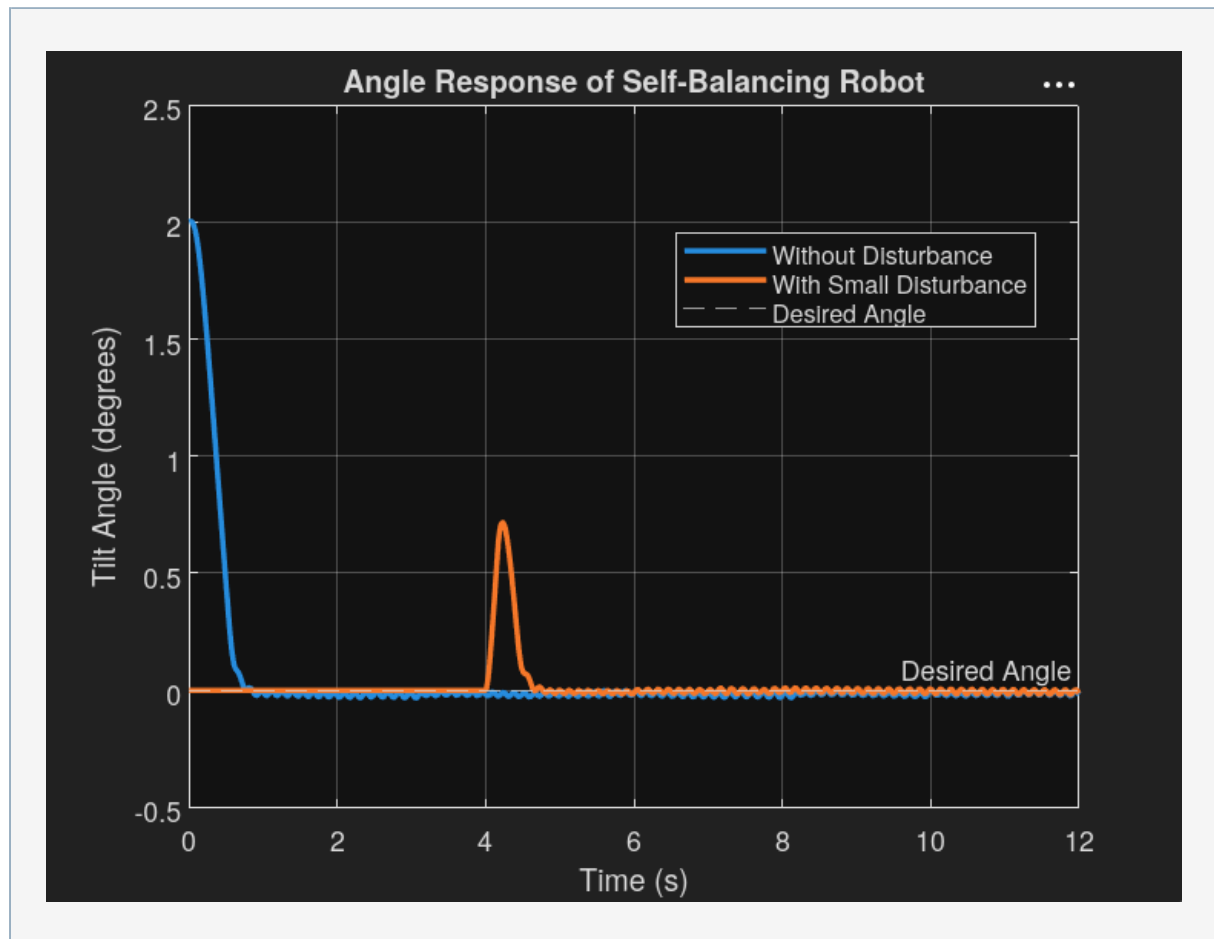


Figure 1: Measured tilt angle $\theta(t)$ during balancing and disturbance rejection (1-minute window).

7.2. PWM / Motor Command Response

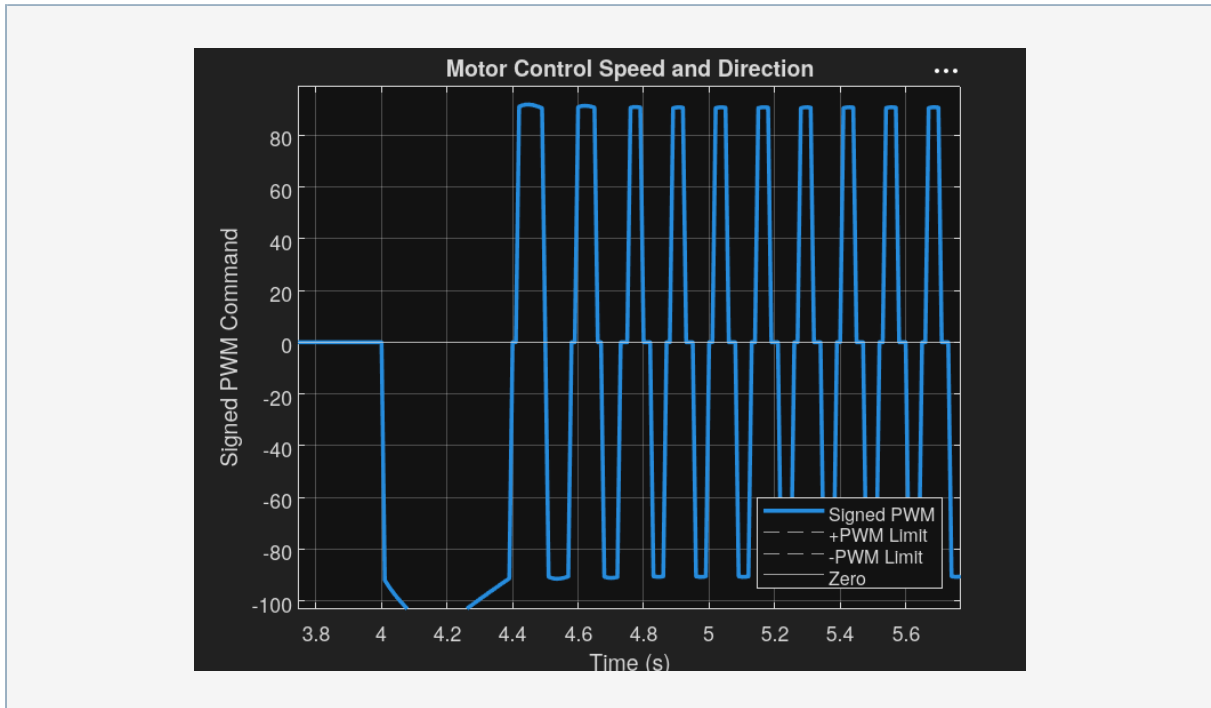


Figure 2: PID control output (PWM motor command) corresponding to the tilt angle response.

7.3. System Disturbance Rejection Response

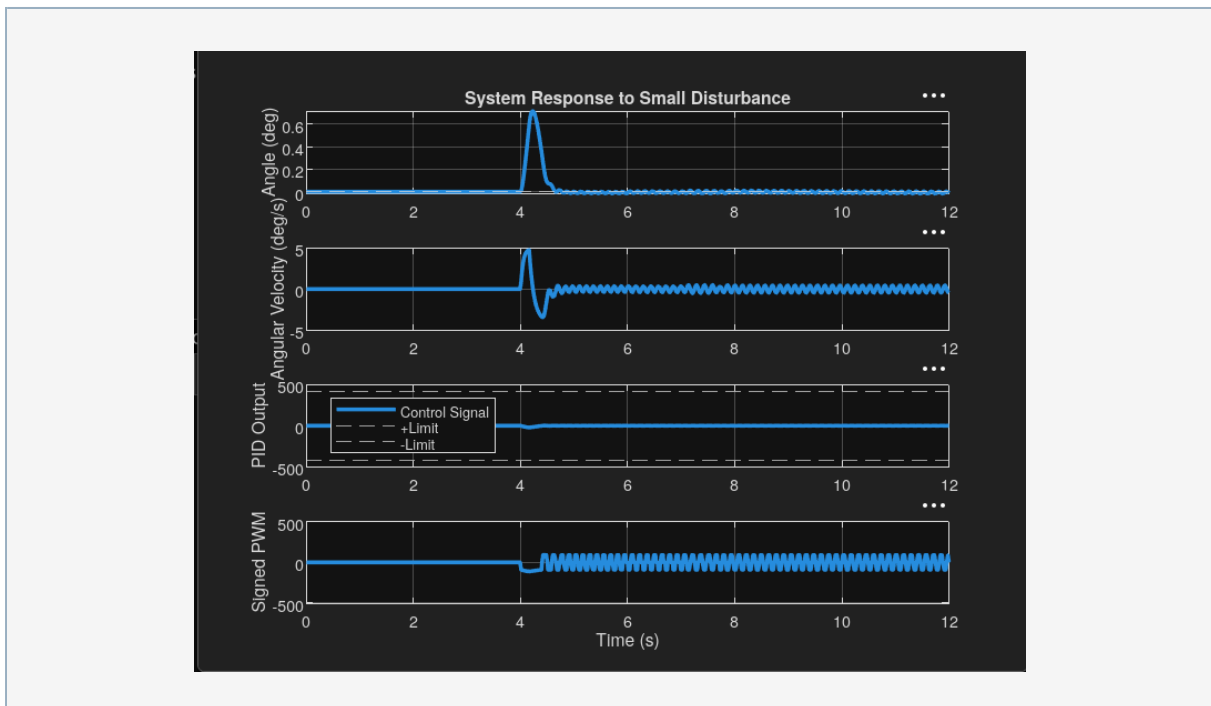


Figure 3: Closed-loop disturbance rejection: angle and motor command during an applied push.

8. Source Code

The complete firmware, written in C++ for the ESP32-S3 using the Arduino framework, is provided below.

```
1  #include <Wire.h>
2  #include <math.h>
3
4  // L298N Motor Driver Pins
5  const int ENA = 4;    const int IN1 = 5;    const int IN2 = 6;
6  const int IN3 = 7;    const int IN4 = 12;   const int ENB = 13;
7
8  // MPU6050 I2C Pins
9  const int S3_SDA = 8;  const int S3_SCL = 9;
10 const int MPU_ADDR = 0x68;
11
12 // Timing
13 unsigned long lastTime = 0;
14 const float dt = 0.01; // 100 Hz
15
16 // Angle
17 float currentAngle = 0.0;
18
19 // PID Gains
20 float Kp = 45.0, Ki = 25.0, Kd = 0.2;
21 float integral = 0.0, prevError = 0.0;
22 const float desiredAngle = 0.0;
23
24 // PWM Settings
25 const int PWM_FREQ = 15000;
26 const int PWM_RESOLUTION = 9;
27 const int MIN_PWM = 90, MAX_PWM = 511;
28
29 void setup() {
30     Serial.begin(115200);
31     pinMode(IN1, OUTPUT); pinMode(IN2, OUTPUT);
32     pinMode(IN3, OUTPUT); pinMode(IN4, OUTPUT);
33     ledcAttach(ENA, PWM_FREQ, PWM_RESOLUTION);
34     ledcAttach(ENB, PWM_FREQ, PWM_RESOLUTION);
35     ledcWrite(ENA, 0); ledcWrite(ENB, 0);
36     Wire.begin(S3_SDA, S3_SCL);
37     Wire.beginTransmission(MPU_ADDR);
38     Wire.write(0x6B); Wire.write(0);
39     Wire.endTransmission(true);
40     delay(1000);
41 }
42
```

```

43 void loop() {
44     unsigned long now = millis();
45     if (now - lastTime >= 10) {
46         lastTime = now;
47
48         // Read MPU6050
49         Wire.beginTransmission(MPU_ADDR); Wire.write(0x3B);
50         Wire.endTransmission(false);
51         Wire.requestFrom(MPU_ADDR, 14, true);
52         int16_t ax = Wire.read() << 8 | Wire.read();
53         int16_t ay = Wire.read() << 8 | Wire.read();
54         int16_t az = Wire.read() << 8 | Wire.read();
55         Wire.read() << 8 | Wire.read(); // Skip Temp
56         int16_t gx = Wire.read() << 8 | Wire.read();
57         int16_t gy = Wire.read() << 8 | Wire.read();
58         int16_t gz = Wire.read() << 8 | Wire.read();
59
60         // Angle Estimation
61         float accAngle = atan2(ay, ax) * 180.0 / PI;
62         float gyroRate = -gz / 131.0;
63         currentAngle = 0.9*(currentAngle + gyroRate*dt) + 0.1*accAngle;
64
65         // PID
66         float error = desiredAngle - currentAngle;
67         integral += error * dt;
68         float derivative = (error - prevError) / dt;
69         float u = Kp*error + Ki*integral + Kd*derivative;
70         prevError = error;
71
72         driveMotors(u);
73     }
74 }
75
76 void driveMotors(float u) {
77     int spd = constrain(MIN_PWM + (int)abs(u), 0, MAX_PWM);
78     if (u > 0) {
79         digitalWrite(IN1, LOW);  digitalWrite(IN2, HIGH);
80         digitalWrite(IN3, LOW);  digitalWrite(IN4, HIGH);
81     } else if (u < 0) {
82         digitalWrite(IN1, HIGH); digitalWrite(IN2, LOW);
83         digitalWrite(IN3, HIGH); digitalWrite(IN4, LOW);
84     } else {
85         digitalWrite(IN1, LOW); digitalWrite(IN2, LOW);
86         digitalWrite(IN3, LOW); digitalWrite(IN4, LOW);
87         spd = 0;
88     }
89     ledcWrite(ENA, spd);  ledcWrite(ENB, spd);

```

90

}

Listing 1: Self-Balancing Robot Firmware (ESP32-S3)

9. Discussion

Complementary Filter Performance. The coefficient $\alpha = 0.9$ (compared to the textbook recommendation of 0.98) was deliberately chosen lower to allow faster drift correction from the accelerometer. Given the relatively high vibration environment of the DC motors, a lower α prevented gyroscope drift from accumulating to destabilizing magnitudes.

Effect of Low K_d . Keeping $K_d = 0.2$ minimal was a conscious decision. A larger derivative term amplifies high-frequency sensor noise, which manifests as jitter in the motor commands. The near-absence of jitter in our robot validated this approach—smooth motor operation reduces mechanical wear and improves balancing consistency.

Integral Windup. With $K_i = 25$, care was taken to ensure the robot was placed in its approximate balance point before the controller was engaged. Integral windup is a concern if the robot is held far from upright for extended periods, as this could saturate the integrator.

PWM Deadband. The `MIN_PWM = 90` floor ensures the motors overcome static friction at all times, preventing the controller from computing a non-zero command that the motors cannot physically execute.

10. Conclusion

A self-balancing two-wheel robot was successfully designed, implemented, and tuned using a PID controller on the ESP32-S3 microcontroller. The GY-MPU6050 IMU was interfaced over I²C, and a complementary filter was employed to produce a reliable real-time tilt angle estimate. The L298N motor driver was driven via PWM at 15 kHz with a 9-bit resolution. After systematic manual tuning ($K_p = 45$, $K_i = 25$, $K_d = 0.2$), the robot demonstrated stable upright balancing for periods exceeding 10 seconds and was capable of rejecting small physical disturbances. The deliberately small derivative gain eliminated motor jitter without compromising damping. The experiment provided practical insight into real-time embedded control of an inherently unstable dynamic system.

References

- [1] UET Lahore Control Systems Lab, *Lab Manual – Lab 12: Self-Balancing Two-Wheel Robot Using PID Control*, 2026.
- [2] Invensense, *MPU-6000 and MPU-6050 Product Specification Revision 3.4*, 2013.
- [3] ST Microelectronics, *L298N Dual Full-Bridge Driver Datasheet*, Rev. 9, 2000.
- [4] Espressif Systems, *ESP32-S3 Technical Reference Manual*, v1.4, 2023.